

The Complete DSA Tutorial - A to Z

- [The Complete DSA Tutorial — A to Z \(Basic to Advanced\)](#)
- [Chapter 0 — Introduction to DSA & Complexity Analysis](#)
 - [0.1 What is a Data Structure? What is an Algorithm?](#)
 - [0.2 Why does DSA even matter? \(The problem it solves\)](#)
 - [0.3 How to read this tutorial](#)
 - [0.4 Big-O, Big-Ω, Big-Θ — measuring “efficiency” precisely](#)
 - [0.5 Space Complexity](#)
 - [0.6 Amortized Analysis \(preview\)](#)
 - [0.7 The 3 Cases: Best, Average, Worst](#)
- [Chapter 3 — Recursion & Backtracking](#)
 - [3.1 What & Why](#)
 - [3.2 Core Theory](#)
 - [3.3 Time & Space Complexity](#)
 - [3.4 Key Patterns](#)
 - [3.5 Problems \(Basic → Medium → Hard\)](#)
- [Chapter 6 — Linked Lists](#)
 - [6.1 What & Why](#)
 - [6.2 Core Theory](#)
 - [6.3 Time & Space Complexity](#)
 - [6.4 Key Patterns](#)
 - [6.5 Problems \(Basic → Medium → Hard\)](#)
- [Chapter 9 — Trees \(Binary Trees\)](#)
 - [9.1 What & Why](#)
 - [9.2 Core Theory & Terminology](#)
 - [9.3 Time & Space Complexity](#)
 - [9.4 Key Patterns](#)
 - [9.5 Problems \(Basic → Medium → Hard\)](#)
- [Chapter 12 — Tries \(Prefix Trees\)](#)
 - [12.1 What & Why](#)
 - [12.2 Core Theory](#)
 - [12.3 Time & Space Complexity](#)
 - [12.4 Key Patterns](#)
 - [12.5 Problems \(Basic → Medium → Hard\)](#)
- [Chapter 15 — Greedy Algorithms](#)

- [15.1 What & Why](#)
- [15.2 Core Theory](#)
- [15.3 Time & Space Complexity](#)
- [15.4 Key Patterns](#)
- [15.5 Problems \(Basic → Medium → Hard\)](#)
- [Chapter 18 — Master Patterns Cheatsheet](#)
 - [18.1 “If you see X in the problem → think Y”](#)
 - [18.2 Complexity Cheat Table \(all structures at a glance\)](#)
 - [18.3 Recursion Recurrence → Big-O Cheat Sheet \(Master Theorem, informal\)](#)
 - [18.4 Interview Problem-Solving Framework \(UMPIRE-style\)](#)

The Complete DSA Tutorial — A to Z (Basic to Advanced)

A sequence-wise guide for high-package placement prep: Theory → Real-life motivation (“why was this invented”) → Time/Space Complexity → Patterns → Graded Problems (Basic → Medium → Hard), for every core DSA topic.

Chapter 0 — Introduction to DSA & Complexity Analysis

0.1 What is a Data Structure? What is an Algorithm?

- **Data Structure (DS):** A way of organizing and storing data in memory so it can be used efficiently. Think of it as *how* you arrange your stuff.
- **Algorithm:** A finite, step-by-step procedure to solve a problem or transform input to output. Think of it as *what steps* you take.

Real-life analogy: A kitchen (memory) needs organization (data structure) — spices in a rack, vegetables in a fridge, knives in a block. The recipe you follow (algorithm) works *with* that organization. A brilliant recipe is slow if your kitchen is a mess; a perfectly organized kitchen is

useless without a recipe. DSA is the study of pairing the right kitchen layout with the right recipe for the job.

0.2 Why does DSA even matter? (The problem it solves)

Early computing problems were solved with whatever ad-hoc code worked. As data grew (millions → billions of records: search engines, social graphs, GPS routes, bank transactions), naive approaches that were “fast enough” for 100 items became unusable for 100 million items. DSA emerged because:

1. **Scale exposes bad choices.** A linear search over 100 items takes microseconds. Over 10 billion web pages, it’s unusable — Google needed hash maps, tries, and inverted indexes.
2. **Different access patterns need different structures.** A phone contacts app needs fast lookup by name (hashing/trees), a music “undo” feature needs LIFO order (stack), a print queue needs FIFO order (queue), GPS navigation needs shortest-path graphs.
3. **Resources are finite.** CPU time and RAM are limited and cost money (cloud bills scale with inefficiency). An $O(n^2)$ algorithm on 1 million records = 10^{12} operations $\approx$ minutes-to-hours; an $O(n \log n)$ version = $\sim 2 \times 10^7$ operations $\approx$ milliseconds.

This is precisely why DSA is the core interview filter for high-paying SDE roles: it proves you can reason about **scale**, not just “make it work on my laptop with 5 sample rows.”

0.3 How to read this tutorial

Every chapter follows the same template: 1. **What & Why** — the concept, and the real problem that forced its invention. 2. **Core theory** — mental model, invariants, how it works internally. 3. **Time & Space Complexity** — for every core operation. 4. **Code skeleton** — canonical implementation (Java/C++-style pseudocode, adapt to your language). 5. **Patterns** — the recurring “shapes” of problems on this topic. 6. **Problems** — Basic → Medium → Hard, with the pattern each teaches.

Work sequentially. Do not skip Arrays/Strings/Recursion — everything else builds on them.

0.4 Big-O, Big-Ω, Big-Θ — measuring “efficiency” precisely

We don’t measure algorithms in seconds (that depends on hardware). We measure how the number of **operations grows** as input size `n` grows. This is **asymptotic analysis**.

- **Big-O (O)** — *upper bound* — “worst case won’t be worse than this.” This is what we quote 95% of the time.
- **Big-Ω (Omega)** — *lower bound* — best case.
- **Big-Θ (Theta)** — *tight bound* — when best and worst are the same order.

Real-life example: You tell a friend “the drive takes at most 2 hours” (upper bound / Big-O), “at least 40 minutes if there’s zero traffic” (lower bound / Big-Ω). If traffic is always predictable, “exactly 1 hour” is Big-Θ.

Rules of thumb for computing Big-O

1. Drop constants: $O(2n) \rightarrow O(n)$.
2. Drop lower-order terms: $O(n^2 + n) \rightarrow O(n^2)$.
3. Different inputs get different variables: comparing two arrays of size n and $m \rightarrow O(n + m)$, not $O(n)$.
4. Nested loops over the same input $\rightarrow$ multiply: two nested loops over $n \rightarrow O(n^2)$.
5. Sequential blocks $\rightarrow$ add, then drop lower order: a loop $O(n)$ followed by another $O(n^2) \rightarrow O(n^2)$.

Common complexity classes (best to worst, growth of operations for $n = 10^6$)

Complexity	Name	Ops @ $n=10^6$	Typical example
$O(1)$	Constant	1	Array index access, hash map get
$O(\log n)$	Logarithmic	~ 20	Binary search, balanced BST ops
$O(n)$	Linear	10^6	Single loop, linear scan
$O(n \log n)$	Linearithmic	$\sim 2 \times 10^7$	Merge sort, heap sort, sort-based algorithms
$O(n^2)$	Quadratic	10^{12}	Nested loops, bubble/insertion sort
$O(n^3)$	Cubic	10^{18}	Naive matrix multiply, 3 nested loops
$O(2^n)$	Exponential	astronomical	Brute-force subsets, naive recursion (fib)
$O(n!)$	Factorial	astronomical	Brute-force permutations, TSP brute force

Practical rule for interviews: if $n \leq 10^8$, aim for $O(n \log n)$ or better; if $n \leq 10^4$, $O(n^2)$ may be fine; if $n \leq 20$, $O(2^n)$ bitmask DP is fine; if $n \leq 10$, $O(n!)$ brute force may pass.

0.5 Space Complexity

Extra memory an algorithm uses *beyond* the input, as a function of n . - **Auxiliary space:** extra space only (excludes input storage) — this is what interviewers usually mean. - Recursion uses **call-stack space** — $O(\text{depth})$ — often overlooked! A recursive function with depth n uses $O(n)$ space even if it allocates no extra variables.

Real-life example: Space complexity is like asking “how much *extra* counter space do I need to cook this recipe,” not “how big is the fridge holding my ingredients” (that’s the input).

0.6 Amortized Analysis (preview)

Some operations are usually cheap but occasionally expensive (e.g., dynamic array resize doubles capacity — most **push** calls are $O(1)$, but one in every n is $O(n)$ to copy). **Amortized cost**

averages this over many operations: dynamic array `push_back` is **$O(1)$ amortized** even though worst-case single call is $O(n)$. This concept recurs in hash tables, dynamic arrays (ArrayList/vector), and union-find with path compression.

0.7 The 3 Cases: Best, Average, Worst

- **Best case:** most favorable input (e.g., already-sorted array for insertion sort $\rightarrow O(n)$).
- **Worst case:** least favorable input (interviewers care about this most, since it's a guarantee).
- **Average case:** expected performance over random inputs (relevant for quicksort: $O(n \log n)$ average, $O(n^2)$ worst).

Next: [Chapter 1 — Arrays](#)

Prev: [Introduction](#) |
Next: [Chapter 2 — Strings](#)

Chapter 2 —
Strings

2.1 What & Why

A **string** is an array of characters, but treated as a distinct type because text has its own operations: comparison, pattern matching, parsing. Strings are usually **immutable** in languages like Java, Python, JS — every “modification” creates a new string, which is why naive string concatenation in a loop is $O(n^2)$ (build with `StringBuilder` / `list` + `join` instead, $O(n)$).

The problem that created dedicated string algorithms:
Searching for a substring naively (check every position, compare every character) is $O(n \cdot m)$. Real systems — search engines, DNA sequence

Next: Chapter 1 — Arrays

matching, spell-checkers, plagiarism detectors — need to search text at massive scale, so specialized algorithms (KMP, Rabin-Karp, Z-algorithm, Tries, Suffix Arrays) were invented to bring this down to $O(n+m)$ or better.

Real-life example:

Ctrl+F in a browser searching a 10,000-word document for “algorithm” — naive char-by-char comparison at every position is wasteful because when a mismatch happens, you *already know* information about the partial match (KMP exploits this to never re-check characters).

2.2 Core Theory

- Strings support **indexing** ($O(1)$) like arrays. -

Immutability cost:

`s += c` in a loop → $O(n)$ per op → $O(n^2)$ total. Fix: mutable buffer

Next: Chapter 1 — Arrays

(StringBuilder/list of chars). - **ASCII vs**

Unicode: character sets matter for hashing and frequency-count array sizing (26 for lowercase, 128/256 for ASCII). -

Palindrome, Anagram, Permutation checks are recurring conceptual building blocks.

2.3 Time & Space Complexity

| Operation |
Complexity | |—|—| |
Access char at index |
 $O(1)$ | |
Concatenation
(immutable) | $O(n)$
per op | | Substring
extraction | $O(k)$ (k =
substring length) | |
Naive substring
search | $O(n \cdot m)$ | |
KMP / Z-algorithm
search | $O(n + m)$ | |
Rabin-Karp (avg) |
 $O(n + m)$; worst
 $O(n \cdot m)$ with hash
collisions | | Sorting
characters | $O(n \log n)$ |

2.4 Key Patterns

Next: [Chapter 1 — Arrays](#)

1. **Two Pointers** — palindrome check, reverse words. 2. **Sliding Window** — longest substring without repeating characters, minimum window substring. 3. **Frequency Count array/HashMap** — anagrams, character counting. 4. **KMP (Knuth-Morris-Pratt)** — build a “failure function” (longest proper prefix that’s also suffix, LPS array) so matching never backtracks in the text — $O(n+m)$. 5. **Rabin-Karp** — rolling hash to compare substrings in $O(1)$ after $O(m)$ preprocessing. 6. **Trie** — prefix-based storage (see Chapter 12). 7. **Manacher’s Algorithm** — longest palindromic substring in $O(n)$.

KMP Failure Function (LPS array)
— code skeleton

2.5 Problems
(Basic → Medium → Hard)

Next: Chapter 1 — Arrays

Basic 1. Reverse a string / reverse words in a sentence. 2. Check palindrome (two pointers). 3. Check anagram of two strings — frequency array, $O(n)$. 4. Count vowels/consonants, char frequency. 5. String to integer (atoi) — edge case handling. 6. Check if a string is a rotation of another (s2 in s1+s1).

Medium 7.

Longest Substring Without Repeating Characters

(LeetCode 3) — sliding window + hashset, $O(n)$. 8.

Longest Palindromic Substring

(LeetCode 5) — expand-around-center $O(n^2)$ or Manacher $O(n)$. 9.

Group Anagrams

(LeetCode 49) — hashmap keyed by sorted string / char-count signature. 10.

Valid Parentheses

(LeetCode 20) — stack (bridges into Chapter 7). 11.

Next: [Chapter 1 — Arrays](#)

String to Integer / Roman to Integer / Integer to Roman

(LeetCode 8, 13, 12).

12. Implement

strStr() (LeetCode 28) — KMP. 13.

Minimum Window

Substring (LeetCode

76) — sliding

window + frequency map, $O(n)$. 14.

Longest Common

Prefix (LeetCode 14).

15. Zigzag

Conversion

(LeetCode 6). 16.

Decode String

(LeetCode 394) — stack-based parsing.

Hard 17.

Regular Expression

Matching (LeetCode

10) — DP, $O(n \cdot m)$.

18. Wildcard

Matching (LeetCode

44) — DP. 19. **Edit**

Distance (LeetCode

72) — DP (bridges into Chapter 14). 20.

Text Justification

(LeetCode 68). 21.

Shortest

Palindrome

(LeetCode 214) — KMP trick. 22.

Longest Valid

Parentheses

(LeetCode 32) —

Next: [Chapter 1 — Arrays](#)

stack/DP, $O(n)$. 23.

Word Break II

(LeetCode 140) —
DP + backtracking.

24. Distinct

Subsequences

(LeetCode 115) —
DP.

Prev: [Arrays](#) | **Next:** [Chapter 3 — Recursion & Backtracking](#)

Chapter 3 — Recursion & Backtracking

3.1 What & Why

Recursion is a function that calls itself on a smaller subproblem until it hits a **base case**. It exists because many problems are naturally **self-similar** — a tree is a node plus two smaller trees; a large factorial is n times a smaller factorial. Trying to express these iteratively can be awkward or require you to manually manage a stack — recursion lets the **call stack** do that bookkeeping for you.

The problem that created it: Traversing nested/hierarchical structures (file systems, org charts, HTML DOM, expression trees) iteratively requires an explicit stack — recursion is the natural language for “solve this by solving smaller versions of itself,” directly modeling divide-and-conquer thinking (used later in Merge Sort, Quick Sort, Binary Search, Tree/Graph traversal, DP).

Backtracking is recursion + **undo**: try a choice, recurse, and if it doesn’t lead to a solution, undo the choice (backtrack) and try the next one. It’s how you exhaustively but *intelligently* explore all possibilities (pruning bad paths early) — used for Sudoku solvers, N-Queens, generating permutations/combinations, and maze-solving robots.

Real-life example: Recursion = Russian nesting dolls (matryoshka) — to open the whole set, you open the outer doll, revealing a smaller doll, and repeat until you hit the smallest (base case). Backtracking = solving a maze by walking a path; whenever you hit a dead end, you retrace your steps to the last junction and try a different corridor, instead of restarting from scratch.

3.2 Core Theory

- Every recursive function needs: (1) **base case** (stops recursion), (2) **recursive case** (calls itself on smaller input), (3) progress toward the base case (or infinite recursion → stack overflow).
- **Call stack**: each call adds a “stack frame” holding local variables & return address. Depth n recursion → $O(n)$ space.
- **Recursion tree**: visualize calls as a tree to compute time complexity — count total nodes/work.
- **Tail recursion**: recursive call is the last operation — some languages optimize this into a loop (Java/Python do NOT optimize this — beware stack overflow for deep recursion).
- **Backtracking = DFS over a decision tree** with pruning: at each node, try each valid choice, recurse, then **undo** (remove from path / unmark visited) before trying the next choice.

3.3 Time & Space Complexity

Concept	Complexity
Factorial / linear recursion	$O(n)$ time, $O(n)$ space (call stack)
Fibonacci naive recursion	$O(2^n)$ time, $O(n)$ space — exponential blowup from recomputation (motivates DP/memoization, Ch. 14)
Fibonacci with memoization	$O(n)$ time, $O(n)$ space
Binary recursion (divide & conquer, e.g. merge sort)	$O(n \log n)$ time typically, $O(\log n)$ space if balanced
Backtracking (permutations)	$O(n \cdot n!)$ time, $O(n)$ space
Backtracking (subsets)	$O(n \cdot 2^n)$ time, $O(n)$ space
Backtracking (N-Queens)	$O(n!)$ worst case, pruned heavily in practice

3.4 Key Patterns

1. **Divide and Conquer**: split into subproblems, solve independently, combine (Merge Sort, Quick Sort, Binary Search).
2. **Choice–Explore–Unchoose** (the backtracking template):

```

void backtrack(State state, List<Choice> choices) {
    if (isGoal(state)) { record(state); return; }
    for (Choice c : choices) {
        if (!isValid(c, state)) continue; // pruning
        state.apply(c); // choose
        backtrack(state, nextChoices); // explore
        state.undo(c); // un-choose
    }
}

```

3. **Memoization**: cache results of subproblems keyed by input to avoid recomputation — turns exponential recursion into polynomial (segues directly into Dynamic Programming, Ch. 14).
4. **Recursion on strings/arrays**: process index `i`, recurse on `i+1`, with an “include/exclude” branch — foundational for subset/DP problems.

3.5 Problems (Basic → Medium → Hard)

Basic

1. Factorial, power (x^n) via recursion — $O(\log n)$ with fast exponentiation.
2. Fibonacci (naive vs memoized).
3. Sum of digits, reverse a number recursively.
4. Check if a string is a palindrome recursively.
5. Print all subsets of a set (power set) — $O(2^n)$.

Medium

6. **Permutations** (LeetCode 46) — backtracking, $O(n \cdot n!)$.
7. **Permutations II** (with duplicates) (LeetCode 47) — backtracking + skip-duplicate logic.
8. **Subsets / Subsets II** (LeetCode 78, 90).
9. **Combination Sum / Combination Sum II** (LeetCode 39, 40).
10. **Generate Parentheses** (LeetCode 22) — backtracking with validity pruning.
11. **Letter Combinations of a Phone Number** (LeetCode 17).
12. **Word Search** (LeetCode 79) — backtracking on grid, DFS + undo visited mark.
13. **Palindrome Partitioning** (LeetCode 131).

Hard

14. **N-Queens** (LeetCode 51) — classic backtracking with column/diagonal pruning, $O(n!)$.
15. **Sudoku Solver** (LeetCode 37) — backtracking with constraint checks.
16. **Word Break II** (backtracking + memo).
17. **Expression Add Operators** (LeetCode 282).
18. **Regular Expression Matching via recursion+memo** (LeetCode 10).
19. **The Knight's Tour** — backtracking on grid.

Prev: [Strings](#) |
Next: [Chapter 4](#)
— [Searching](#)

Prev: [Recursion & Backtracking](#) |
Next: [Chapter 5](#)
— [Sorting](#)

Chapter 5 —
Sorting

5.1 What &
Why

Sorting arranges elements in a defined order. It's foundational because **many algorithms become dramatically simpler/faster once data is sorted**: binary search needs sorted data, duplicate detection becomes a single pass, merge-based algorithms rely on sorted merges, scheduling/greedy algorithms sort by deadline/ratio first.

The problem that created it: Before computers, sorting punch

cards or ledgers
by hand was
tedious and error-
prone (early
“sorting” = filing
cabinets, library
card catalogs
alphabetized for
 $O(\log n)$ -ish
lookup by a
human).
Computer science
needed provably
efficient,
comparison-based
general-purpose
sorting — leading
to the
mathematical
result that
**comparison sorts
cannot beat $O(n \log n)$** in the
worst case
(proven via
decision-tree
argument: $n!$
possible orderings
need $\log_2(n!) \approx n \log n$
comparisons
to distinguish).

Real-life

example: Sorting
exam scripts by
roll number
before
distributing them
— once sorted, a

teacher can find any student's paper via binary search instead of shuffling through a random pile.

5.2 The Sorting Algorithms

| Algorithm | Best |
Average | Worst |
Space | Stable? |
Notes | |—|—|—|
—|—|—|—| |
Bubble Sort | $O(n)$
| $O(n^2)$ | $O(n^2)$ |
 $O(1)$ | Yes |
Repeatedly swaps
adjacent out-of-
order pairs | |
Selection Sort |
 $O(n^2)$ | $O(n^2)$ |
 $O(n^2)$ | $O(1)$ | No |
Repeatedly selects
min and places it |
| Insertion Sort |
 $O(n)$ | $O(n^2)$ | $O(n^2)$
| $O(1)$ | Yes | Builds
sorted prefix one
element at a time;
great for nearly-
sorted/small data |
| Merge Sort | $O(n \log n)$ | $O(n \log n)$
| $O(n \log n)$ | $O(n)$ |
Yes | Divide &
conquer;
guarantees $O(n \log n)$ always | |

Prev: [Strings](#) |
Next: [Chapter 4](#)
[— Searching](#)

Quick Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$ | $O(\log n)$ | No | Divide & conquer via pivot partition; fastest in practice (cache-friendly, in-place) |
| Heap Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$ | No | Build max-heap, repeatedly extract max | |
Counting Sort | $O(n+k)$ | $O(n+k)$ | $O(n+k)$ | $O(k)$ | Yes | Non-comparison; needs small integer range k | |
Radix Sort | $O(d \cdot (n+k))$ | $O(d \cdot (n+k))$ | $O(d \cdot (n+k))$ | $O(n+k)$ | Yes | Non-comparison; sorts digit by digit | |
| Bucket Sort | $O(n+k)$ | $O(n+k)$ | $O(n^2)$ | $O(n)$ | Yes | Distributes into buckets, sorts each |

Stability matters when sorting objects by one key but wanting to preserve

relative order of equal keys (e.g., sort employees by department, then within a stable sort, original name order within a department is preserved).

5.3 Core Theory — How the Big Ones Work

- **Merge Sort:** split array into halves recursively until size 1, then merge two sorted halves in $O(n)$ using a temp array. Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ by the Master Theorem. Always $O(n \log n)$ regardless of input — great when worst-case guarantees matter (external sorting, linked lists). - **Quick Sort:** pick a **pivot**, partition array so smaller elements go left, larger go right, recurse on both

sides. Average $O(n \log n)$; worst case $O(n^2)$ happens when pivot choice is consistently bad (e.g., always picking first element on an already-sorted array) —

mitigated with **random pivot** or **median-of-three**.

- **Heap Sort:** build a max-heap (Ch. 11) in $O(n)$, then repeatedly swap root (max) to the end and sift down — $O(n \log n)$, $O(1)$ extra space (in-place), but not stable and worse cache locality than quicksort. -

Counting/Radix

Sort: exploit known, bounded value ranges to sort **without comparisons**, breaking the $O(n \log n)$ comparison-sort lower bound — used when sorting small integers, digit-limited numbers, or fixed-

length strings
(e.g., sorting
phone numbers,
sorting scores 0-
100).

5.4 When to
use what
(interview
heuristic)

- Need
guaranteed $O(n \log n)$ & stability
& don't mind
extra memory →
Merge Sort (also
the go-to for
sorting linked lists,
and for external
sorting of data
too big for RAM).
- Need fastest
average speed, in-
place, don't care
about worst case
→ **Quick Sort**
(with randomized
pivot).
- Need
 $O(1)$ extra space
and $O(n \log n)$
worst case
guarantee →
Heap Sort.
- Data
is integers in a
small known
range →
Counting Sort.
- Small or nearly-
sorted array (e.g.,

Prev: [Strings](#) |
Next: [Chapter 4](#)
— [Searching](#)

$n < 20$, or
insertion into an
already sorted
array) → **Insertion
Sort.**

5.5 Problems
(Basic → Medium
→ Hard)

Basic 1.
Implement
Bubble, Selection,
Insertion sort from
scratch. 2.
Implement Merge
Sort and Quick
Sort from scratch.
3. Sort an array of
0s, 1s, and 2s
(Dutch National
Flag) (LeetCode
75) — $O(n)/O(1)$.
4. Find Kth
largest/smallest
using sorting.

Medium 5.
**Merge Sort on
Linked List**
(LeetCode 148) —
 $O(n \log n)$. 6. **Sort
Colors** (LeetCode
75). 7. **Kth
Largest Element
in an Array**
(LeetCode 215) —
Quickselect,
average $O(n)$. 8.
Merge Intervals

Prev: [Strings](#) |
Next: [Chapter 4](#)
[— Searching](#)

(LeetCode 56) —
sort by start. 9.

Largest Number

(LeetCode 179) —
custom
comparator sort.

10. Meeting

Rooms II

(LeetCode 253) —
sort + min-heap.

11. Relative Sort

Array (LeetCode
1122) — counting
sort variant. 12. **H-
Index** (LeetCode
274) — sort +
scan.

Hard 13.

Count of Smaller Numbers After Self

(LeetCode
315) — merge
sort with count,
 $O(n \log n)$. 14.

Reverse Pairs

(LeetCode 493) —
modified merge
sort. 15.

Maximum Gap

(LeetCode 164) —
bucket sort / radix
sort, $O(n)$. 16.

Chalkboard

Median-based

scheduling

problems — sort
+ greedy. 17.

External Sort

Prev: [Strings](#) |
Next: [Chapter 4](#)
— [Searching](#)

simulation

(theory) — merge
sort applied to
disk-based data
larger than RAM.

Prev: [Searching](#) | **Next:** [Chapter 6 — Linked List](#)

Chapter 6 — Linked Lists

6.1 What & Why

A **Linked List** stores elements as nodes, each holding data + a pointer/reference to the next node. Unlike arrays, nodes are **not contiguous** in memory.

The problem that created it: Arrays are great for random access but terrible for frequent insertion/deletion in the middle — shifting elements costs $O(n)$. Early systems (like early Lisp, and OS-level free memory lists) needed a structure where **inserting/deleting a known node is $O(1)$** — you just re-point a few pointers instead of shifting a whole block. This is why OS memory allocators, browser history (doubly linked list), music playlists (next/prev), and undo-chains use linked lists.

Real-life example: A treasure hunt where each clue tells you where to find the next clue — you can't jump straight to clue #7 (no random access, must walk from the start), but if you want to insert a new clue between #3 and #4, you just change what clue #3 points to — no need to renumber/shift anything, unlike inserting a new page into a bound book (array).

6.2 Core Theory

- **Singly Linked List:** each node → next node only. Traversal is one-directional.
- **Doubly Linked List:** each node has `prev` and `next` — allows $O(1)$ backward traversal and easier deletion (no need to track previous node separately). Costs extra memory per node.
- **Circular Linked List:** tail points back to head — useful for round-robin scheduling, circular buffers.

- **Dummy/Sentinel Node:** a fake head node used to simplify edge cases (empty list, deleting head) in code — extremely common trick.
- **Fast & Slow Pointers (Floyd's Tortoise and Hare):** two pointers moving at different speeds to detect cycles, find the middle, or find the Nth-from-end node — $O(n)$ time, $O(1)$ space, avoiding a second pass or extra storage.

6.3 Time & Space Complexity

Operation	Array	Singly Linked List	Doubly Linked List
Access by index	$O(1)$	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$
Insert at head	$O(n)$	$O(1)$	$O(1)$
Insert at tail (with tail pointer)	$O(1)$ amortized	$O(1)$	$O(1)$
Insert/Delete in middle (given node ref)	$O(n)$	$O(1)^*$	$O(1)$
Delete at head	$O(n)$	$O(1)$	$O(1)$
Space per element	$O(1)$	$O(1)$ + pointer	$O(1)$ + 2 pointers

* $O(1)$ only if you already have a reference to the node before it (singly) — otherwise $O(n)$ to find it.

6.4 Key Patterns

1. **Dummy Head Node** — simplifies insert/delete-at-head edge cases.
2. **Fast & Slow Pointers** — cycle detection (Floyd's), find middle node, find Nth node from end.
3. **Reverse a Linked List (iterative in-place)** — three pointers: `prev`, `curr`, `next`.
4. **In-place merge of two sorted lists** — pointer manipulation, no extra space.
5. **Recursive Linked List processing** — many list problems have elegant recursive solutions (reverse, merge, palindrome check) but cost $O(n)$ stack space vs $O(1)$ iterative.

Code skeleton — Reverse a Linked List

```

ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

```

Code skeleton — Floyd's Cycle Detection

```

boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

```

6.5 Problems (Basic → Medium → Hard)

Basic

1. Reverse a linked list (LeetCode 206) — iterative & recursive.
2. Find the middle of a linked list (LeetCode 876) — fast/slow pointers.
3. Detect a cycle (LeetCode 141) — Floyd's algorithm.
4. Find the Nth node from the end — two pointers with gap of N.
5. Delete a node given only access to that node (LeetCode 237).
6. Merge two sorted linked lists (LeetCode 21).

Medium

7. **Detect Cycle Start Point** (LeetCode 142) — Floyd's + math proof of meeting point.
8. **Remove Nth Node From End of List** (LeetCode 19).
9. **Reorder List** (LeetCode 143) — find middle + reverse second half + merge.
10. **Add Two Numbers (as linked lists)** (LeetCode 2).
11. **Copy List with Random Pointer** (LeetCode 138) — hashmap or interleaving trick.
12. **Rotate List** (LeetCode 61).
13. **Swap Nodes in Pairs** (LeetCode 24).
14. **Palindrome Linked List** (LeetCode 234) — find middle + reverse + compare.
15. **Intersection of Two Linked Lists** (LeetCode 160) — two-pointer switch-list trick, $O(n+m)/O(1)$.

Hard

16. **Merge K Sorted Lists** (LeetCode 23) — min-heap or divide & conquer merge, $O(N \log k)$.
17. **Reverse Nodes in k-Group** (LeetCode 25).
18. **LRU Cache** (LeetCode 146) — doubly linked list + hashmap, $O(1)$ get/put (bridges to Ch. 8 Hashing).
19. **LFU Cache** (LeetCode 460) — advanced doubly-linked-list + hashmap design.
20. **Flatten a Multilevel Doubly Linked List** (LeetCode 430).

Prev: [Sorting](#) |
Next: [Chapter 7](#) —
[Stacks & Queues](#)

Prev: [Linked Lists](#) |
Next: [Chapter 8](#) —
[Hashing](#)

Chapter 8 —
Hashing

8.1 What & Why

Hashing maps a key to an array index via a **hash function**, giving average $O(1)$ insert/search/delete — the fastest lookup possible without extra structure like sorting.

The problem that created it: Arrays give $O(1)$ access *only if you already know the index*. Real-world keys are not small integers — they're names, emails, IP addresses, arbitrary objects. Searching a sorted array is $O(\log n)$; linear array is $O(n)$. Hashing was invented to get **$O(1)$ average lookup for**

arbitrary keys by converting the key into an array index via a hash function — this is the backbone of database indexes, caches, compilers' symbol tables, and de-duplication systems.

Real-life example:

A library that shelves books not alphabetically but by a computed “shelf number” derived from the ISBN (hash function) — you compute the shelf number and walk straight there, instead of scanning every shelf (array) or doing a binary search through an alphabetized catalog (BST).

8.2 Core Theory

- Hash Function:

converts key → index (`index = hash(key) % table_size`).

Good hash

functions distribute keys uniformly to minimize collisions.

- **Collision:** two different keys map to the same index. Since key space $\gg$ table size, collisions are inevitable (Pigeonhole Principle) — good design handles them well, not avoids them.

Collision

Resolution: -

- **Chaining:** each bucket holds a linked list (or tree, in Java 8+ HashMap when a bucket gets large) of all keys hashing there.

- **Open Addressing:** on collision, probe another slot (linear probing, quadratic probing, double hashing) — keeps all data in-place, better cache locality, but needs careful deletion handling (tombstones).

Load Factor ($\alpha = n/\text{table_size}$):

Prev: [Sorting](#) |
Next: [Chapter 7](#)
[— Stacks & Queues](#)

when it exceeds a threshold (commonly 0.75), the table **resizes** (rehashes all keys into a bigger table) to maintain $O(1)$ average performance — this resizing is why insert is **$O(1)$ amortized**, not strictly $O(1)$. -

HashSet vs

HashMap: HashSet = HashMap where you only care about key existence, not an associated value.

8.3 Time & Space Complexity

Operation	Average	Worst Case (bad hash/many collisions)
Insert	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$
Space	$O(n)$	$O(n)$

Worst case $O(n)$ happens if the hash function is poor and everything collides into one

Prev: [Sorting](#) |
Next: [Chapter 7](#)
[— Stacks & Queues](#)

bucket (degrades to a linked list scan)
— Java's HashMap mitigates this since Java 8 by treeifying long chains ($O(\log n)$ worst case per bucket).

8.4 Key Patterns

1. **Frequency**

Counting — count occurrences of elements (anagrams, majority element, mode). 2.

Existence Check / Deduplication —

“have I seen this before?” in $O(1)$, turning $O(n^2)$ brute-force pair-checking into $O(n)$.

3. **Two Sum**

pattern — store value → index

while scanning, check `target - current` exists,

$O(n)$. 4. **Grouping**

by computed key

— group anagrams (key = sorted string), group by remainder (key = `sum % k`). 5.

Prefix Sum + Hashmap —

Prev: [Sorting](#) |
Next: [Chapter 7](#)
— [Stacks &
Queues](#)

subarray sum
problems: store
prefixSum →
count/index to
find subarrays
summing to k in
 $O(n)$ (Ch. 1,
problem 15). 6.

**HashMap as a
Graph Adjacency
List** — foundational
for Graph
representation (Ch.
13).

Code skeleton
— Two Sum with
HashMap

8.5 Problems
(Basic → Medium →
Hard)

Basic 1. Two
Sum (LeetCode 1)
— $O(n)$. 2. Check
for duplicates in an
array (LeetCode
217). 3. First non-
repeating character
in a string
(LeetCode 387). 4.
Valid Anagram
(LeetCode 242). 5.
Intersection of Two
Arrays (LeetCode
349). 6. Majority
Element (LeetCode
169) — hashmap,

Prev: [Sorting](#) |
Next: [Chapter 7](#)
[— Stacks & Queues](#)

or Boyer-Moore
voting for $O(1)$
space.

Medium 7.

Group Anagrams

(LeetCode 49). 8.

Subarray Sum

Equals K (LeetCode
560) — prefix sum
+ hashmap, $O(n)$. 9.

Longest

Consecutive

Sequence

(LeetCode 128) —
hashset, $O(n)$. 10.

4Sum II (LeetCode
454) — split into
two pair-sum
hashmaps, $O(n^2)$.

11. Top K Frequent Elements

(LeetCode 347) —
hashmap +
heap/bucket sort.

12. Copy List with Random Pointer

(LeetCode 138) —
hashmap for node
mapping. 13.

Design a HashMap from scratch

(LeetCode 706) —
implement
chaining/open
addressing. 14.

Isomorphic

Strings (LeetCode

Prev: [Sorting](#) |
Next: [Chapter 7](#)
— [Stacks &
Queues](#)

205). 15. **LRU**

Cache (LeetCode 146) — hashmap + doubly linked list.

Hard 16.

Substring with Concatenation of All Words

(LeetCode 30) — hashmap + sliding window. 17.

Longest Substring with At Most K Distinct Characters

(LeetCode 340) — hashmap + sliding window. 18. **Insert**

Delete

GetRandom O(1)

(LeetCode 380) — hashmap + array combo. 19. **All**

O`one Data Structure

(LeetCode 432) — hashmap + doubly linked list, advanced design.

20. **Design Twitter**

(LeetCode 355) — hashmap + heap combo, system-design-flavored DSA.

Chapter 9 — Trees (Binary Trees)

9.1 What & Why

A **Tree** is a hierarchical structure: a root node with child nodes, each child itself the root of a subtree, with no cycles.

The problem that created it: Linear structures (arrays, linked lists) can't naturally represent **hierarchical relationships** — a file system (folders containing folders), an org chart (manager → reports), an HTML DOM (elements nested in elements), a decision process (yes/no branches). Trees generalize this: $O(\log n)$ search *and* natural hierarchy representation, which linked lists lack and arrays fake awkwardly.

Real-life example: A company org chart — the CEO (root) has VPs (children), who have directors (grandchildren), who have managers, etc. Asking "how many people report up through this VP" is a tree traversal. A file explorer's folder tree is the same idea.

9.2 Core Theory & Terminology

- **Root, Parent, Child, Leaf (no children), Height (longest path root→leaf), Depth (distance from root).**
- **Binary Tree:** each node has at most 2 children (left, right) — no ordering constraint (that's BST, Ch. 10).
- **Full Binary Tree:** every node has 0 or 2 children.
- **Complete Binary Tree:** all levels full except possibly the last, filled left to right (basis of array-based Heaps, Ch. 11).
- **Balanced Tree:** height is $O(\log n)$ — guarantees efficient operations (vs a degenerate "linked-list-like" tree with height $O(n)$).
- **Traversals:**
 - **DFS-based:** Preorder (root, left, right), Inorder (left, root, right), Postorder (left, right, root) — typically recursive, $O(n)$ time, $O(h)$ space (h = height, due to call stack).
 - **BFS-based:** Level-order — uses a queue, $O(n)$ time, $O(w)$ space (w = max width).

9.3 Time & Space Complexity

Operation	Balanced Tree	Skewed/Degenerate Tree
Search	$O(\log n)$	$O(n)$
Insert/Delete	$O(\log n)$	$O(n)$
Traversal (any)	$O(n)$	$O(n)$
Space (traversal recursion)	$O(\log n)$	$O(n)$
Space (structure)	$O(n)$	$O(n)$

9.4 Key Patterns

1. **Recursive Divide & Conquer on Trees** — solve for left subtree, right subtree, combine at root (height, diameter, balanced-check, sum problems).
2. **DFS with path tracking** — root-to-leaf path problems (path sum, all paths).
3. **BFS / Level-order** — level-by-level processing (zigzag traversal, right-side view, level averages).
4. **Two-node relationship problems** — LCA (Lowest Common Ancestor), diameter, distance between nodes.
5. **Serialization/Deserialization** — encode a tree to a string and back — tests full traversal understanding.
6. **Morris Traversal** — $O(1)$ space inorder traversal using temporary thread pointers (advanced).

Code skeleton — Traversals

```

void inorder(Node root) {
    if (root == null) return;
    inorder(root.left);
    visit(root);
    inorder(root.right);
}

void levelOrder(Node root) {
    if (root == null) return;
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node cur = q.poll();
            visit(cur);
            if (cur.left != null) q.add(cur.left);
            if (cur.right != null) q.add(cur.right);
        }
    }
}

```

Code skeleton — Height / Diameter pattern

```

int height(Node root) {
    if (root == null) return 0;
    return 1 + Math.max(height(root.left), height(root.right));
}

```

9.5 Problems (Basic → Medium → Hard)

Basic

1. Preorder, Inorder, Postorder traversal (recursive & iterative with explicit stack).
2. Level Order Traversal (LeetCode 102) — BFS.
3. Maximum Depth of Binary Tree (LeetCode 104).

4. Check if two trees are identical (LeetCode 100).
5. Invert/Mirror a Binary Tree (LeetCode 226).
6. Count leaf nodes / count total nodes.

Medium

7. **Balanced Binary Tree check** (LeetCode 110).
8. **Diameter of Binary Tree** (LeetCode 543) — combine subtree heights.
9. **Path Sum I & II** (LeetCode 112, 113).
10. **Lowest Common Ancestor of a Binary Tree** (LeetCode 236) — $O(n)$.
11. **Zigzag Level Order Traversal** (LeetCode 103).
12. **Right Side View** (LeetCode 199).
13. **Construct Binary Tree from Preorder and Inorder Traversal** (LeetCode 105).
14. **Symmetric Tree** (LeetCode 101).
15. **Flatten Binary Tree to Linked List** (LeetCode 114).
16. **Vertical Order Traversal** (LeetCode 987).
17. **Sum Root to Leaf Numbers** (LeetCode 129).

Hard

18. **Binary Tree Maximum Path Sum** (LeetCode 124) — global max tracked during recursion.
19. **Serialize and Deserialize Binary Tree** (LeetCode 297).
20. **Distinct Subsequences / advanced DP-on-tree problems.**
21. **Binary Tree Cameras** (LeetCode 968) — greedy + tree DP.
22. **House Robber III** (LeetCode 337) — tree DP (bridges into Ch. 14).
23. **Count Nodes in Complete Binary Tree** (LeetCode 222) — $O((\log n)^2)$ exploiting completeness.

Prev: [Hashing](#) |
Next: [Chapter 10](#)
[— Binary Search](#)
[Trees](#)

Prev: [Trees](#) | **Next:**
[Chapter 11 — Heaps](#)
[& Priority Queues](#)

Chapter 11 —
Heaps & Priority
Queues

11.1 What & Why

A **Heap** is a complete binary tree (Ch. 9) stored in an array, satisfying the **heap property**: in a Max-Heap, every parent $\geq$ its children (root = maximum); in a Min-Heap, every parent $\leq$ its children (root = minimum).

The problem that created it: You often need repeated access to “the current maximum/minimum” from a changing collection — task schedulers need “highest priority task next,” Dijkstra’s algorithm needs “closest unvisited node next,” event simulators need “next event in time order.” Sorting the

Prev: [Hashing](#) |
Next: [Chapter 10](#)
[— Binary Search](#)
[Trees](#)

whole collection
every time is $O(n \log n)$ per query —
wasteful. A **full BST**
gives $O(\log n)$ find-
min but overkill
(needs full ordering
+ more memory for
pointers). The heap
gives exactly what's
needed — $O(1)$ peek
at the extreme value,
 $O(\log n)$
insert/remove —
using a compact
array (no pointers
needed, since a
complete binary
tree's children are at
predictable indices
 $2i+1, 2i+2$).

Real-life example: A
hospital emergency
room triage queue
— patients aren't
served first-come-
first-served (queue)
but by severity
(priority). The "most
critical patient" must
always be quickly
identifiable and
extractable as new
patients arrive
continuously —
exactly what a

Prev: [Hashing](#) |
Next: [Chapter 10](#)
— [Binary Search](#)
[Trees](#)

priority queue (heap)
is built for.

11.2 Core Theory

- **Array**

representation: for
node at index i ,
children are at
 $2i+1$ (left) and
 $2i+2$ (right); parent
is at $(i-1)/2$. No
pointers needed —
pure index math,
thanks to the
“complete tree”

shape. - **Heapify**
(sift-down/sift-up):
restore heap
property after insert
(sift-up from the
bottom) or removal
(sift-down from the
root) — $O(\log n)$
each. - **Build-Heap:**

converting an
arbitrary array into a
heap takes $O(n)$, not
 $O(n \log n)$ — a
classic surprising
result from careful
amortized analysis
(most nodes are
near the bottom,
needing little sifting).

- **Priority Queue** is
the *abstract concept*
("give me the
highest priority item

next”); **Heap** is the *typical implementation* of it (others exist: Fibonacci heap, pairing heap — used in advanced graph algorithms for better amortized complexity).

11.3 Time & Space Complexity

Operation
Complexity — —
Peek min/max $O(1)$
Insert $O(\log n)$
Extract min/max
$O(\log n)$ Build
heap from array
$O(n)$ Heap Sort
$O(n \log n)$ Search
arbitrary element
$O(n)$ (heaps are NOT
designed for search)
Space $O(n)$

11.4 Key Patterns

1. **Top-K problems**
 — maintain a heap of size K (min-heap for “ K largest,” max-heap for “ K smallest”) — $O(n \log k)$, better than sorting the full array $O(n \log n)$ when $k \ll n$. 2. **Two Heaps**

Prev: [Hashing](#) |
Next: [Chapter 10](#)
— [Binary Search](#)
[Trees](#)

(median

maintenance) — a
max-heap for the
smaller half + min-
heap for the larger
half, keeping them
balanced — $O(\log n)$
insert, $O(1)$ median
lookup. 3. **Merge K**

Sorted Lists/Arrays

— min-heap of size
K holding the
current front of each
list, $O(N \log K)$. 4.

Greedy + Heap —

task scheduling,
meeting rooms,
always
process/compare
against the current
best via a heap. 5.

Heap as Priority

Queue in Graph

Algorithms —

Dijkstra's shortest
path, Prim's MST
(both in Ch. 13) rely
fundamentally on
extracting
"closest/cheapest
next" in $O(\log n)$.

Code skeleton

— using a
language's built-in
heap (Java
PriorityQueue)

Prev: [Hashing](#) |
Next: [Chapter 10](#)
— [Binary Search](#)
[Trees](#)

11.5 Problems
(Basic → Medium →
Hard)

Basic 1.
Implement a Min-Heap and Max-Heap from scratch (array-based, with sift-up/sift-down). 2. Kth Largest Element in a Stream (LeetCode 703). 3. Last Stone Weight (LeetCode 1046). 4. Heap Sort implementation.

Medium 5. **Kth Largest Element in an Array** (LeetCode 215) — heap $O(n \log k)$ or Quickselect $O(n)$ avg. 6. **Top K Frequent Elements** (LeetCode 347) — hashmap + heap. 7. **K Closest Points to Origin** (LeetCode 973). 8. **Merge K Sorted Lists** (LeetCode 23) — $O(N \log K)$. 9. **Task Scheduler** (LeetCode 621) — greedy + heap. 10. **Reorganize String** (LeetCode 767) — greedy + max-heap. 11. **Meeting Rooms**

Prev: [Hashing](#) |
Next: [Chapter 10](#)
— [Binary Search](#)
[Trees](#)

II (LeetCode 253) — min-heap of end times. 12. **Ugly Number II** (LeetCode 264) — heap or 3-pointer DP.

Hard 13. **Find Median from Data Stream** (LeetCode 295) — two-heap technique, $O(\log n)$ insert, $O(1)$ median. 14. **Sliding Window Median** (LeetCode 480) — two heaps + lazy deletion. 15. **Trapping Rain Water II** (LeetCode 407) — min-heap + BFS on grid boundary. 16. **IPO / Maximize Capital** (LeetCode 502) — greedy + two heaps. 17. **Smallest Range Covering Elements from K Lists** (LeetCode 632) — min-heap across K lists. 18. **Design Twitter** (LeetCode 355) — heap-based feed merge.

Chapter 12 — Tries (Prefix Trees)

12.1 What & Why

A **Trie** (from **retrieval**) is a tree where each edge represents a character, and each root-to-node path represents a string prefix. Words sharing a prefix share the same path down the tree.

The problem that created it: Autocomplete (Google search suggestions, IDE code completion, phone keyboard predictions) needs “give me all words starting with prefix X” *fast*, and repeatedly, for prefixes typed character by character. A hashmap of all words gives $O(1)$ exact lookup but **cannot efficiently answer prefix queries** — you’d have to scan every word checking `startsWith()`, $O(n \cdot m)$. A sorted array/BST of words gives $O(\log n + k)$ prefix range but is complex to maintain. A Trie gives $O(m)$ prefix lookup (m = prefix length, independent of how many words are stored!) by literally walking down shared prefix paths, and naturally deduplicates shared prefixes to save space (e.g., “cat,” “car,” “card” share the “ca” path).

Real-life example: A dictionary’s physical structure if organized as a decision tree — to find all words starting with “pre”, you walk $p \rightarrow r \rightarrow e$ and everything below that node is your answer set, without needing to touch any word that doesn’t share that prefix path.

12.2 Core Theory

- Each **Trie node** holds: an array/map of children (indexed by character, e.g. 26 slots for lowercase English), and a boolean flag `isEndOfWord`.
- **Insert:** walk/create nodes character by character, mark the last node `isEndOfWord = true`.
- **Search (exact word):** walk character by character; word exists if path exists AND last node’s `isEndOfWord` is true.
- **StartsWith (prefix search):** walk character by character; prefix exists if the path exists (regardless of `isEndOfWord`).
- **Space tradeoff:** a Trie can use more memory than a hashset of the same words (many small node objects with pointer arrays) — this is the cost paid for $O(m)$ prefix operations.

12.3 Time & Space Complexity

Let `m` = length of the word/prefix being processed, `n` = number of words, `$\Sigma$` = alphabet size.

Operation	Complexity
Insert a word	$O(m)$
Search exact word	$O(m)$
Search prefix (startsWith)	$O(m)$
Space	$O(n \cdot m \cdot \Sigma)$ worst case (shared prefixes reduce this in practice)

Compare to a HashSet: $O(1)$ average exact search, but **cannot** do prefix search without $O(n)$ scan — this is the Trie’s whole value proposition.

12.4 Key Patterns

1. **Autocomplete / Prefix matching** — walk to the prefix node, then DFS the subtree to collect all completions.
2. **Word Search on a Grid with a Dictionary** — build a Trie of the dictionary, then DFS the grid, pruning any path whose prefix isn’t in the Trie (turns brute-force exponential search into something tractable).
3. **Wildcard / Fuzzy Matching in a Trie** — DFS with backtracking, treating `.` / wildcard as “try all children”.
4. **Bitwise Trie (Binary Trie)** — a Trie over the *bits* of numbers (32 levels for 32-bit ints) — used for Maximum XOR pair problems (bridges to Ch. 16 Bit Manipulation).
5. **Trie + DP** — Word Break style problems combine Trie prefix lookup with DP over string indices.

Code skeleton — Trie

```

class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord = false;
}

class Trie {
    TrieNode root = new TrieNode();

    void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new
TrieNode();
            node = node.children[idx];
        }
        node.isEndOfWord = true;
    }

    boolean search(String word) {
        TrieNode node = traverse(word);
        return node != null && node.isEndOfWord;
    }

    boolean startsWith(String prefix) {
        return traverse(prefix) != null;
    }

    private TrieNode traverse(String s) {
        TrieNode node = root;
        for (char c : s.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return null;
            node = node.children[idx];
        }
        return node;
    }
}

```

12.5 Problems (Basic → Medium → Hard)

Basic

1. Implement Trie (Prefix Tree) (LeetCode 208).
2. Count words with a given prefix.
3. Longest common prefix of a set of strings using a Trie.

Medium

4. **Design Add and Search Words Data Structure** (LeetCode 211) — Trie + wildcard DFS.
5. **Replace Words** (LeetCode 648) — Trie prefix matching for stemming.
6. **Map Sum Pairs** (LeetCode 677).
7. **Implement Magic Dictionary** (LeetCode 676).
8. **Word Break** (LeetCode 139) — can be solved with Trie + DP.

Hard

9. **Word Search II** (LeetCode 212) — Trie + grid DFS backtracking, classic hard-tier combo.
10. **Maximum XOR of Two Numbers in an Array** (LeetCode 421) — Binary Trie over bits, $O(n \cdot 32)$.
11. **Palindrome Pairs** (LeetCode 336) — Trie of reversed words.
12. **Stream of Characters** (LeetCode 1032) — reverse Trie + streaming matching.
13. **Short Encoding of Words** (LeetCode 820) — Trie built from suffixes.

Prev: [Heaps & Priority Queues](#) |
Next: [Chapter 13 — Graphs](#)

Prev: [Tries](#) | **Next:** [Chapter 14 — Dynamic Programming](#)

Chapter 14 —
Dynamic
Programming (DP)

14.1 What &
Why

Dynamic Programming
solves problems
by breaking them
into overlapping
subproblems,
solving each
subproblem **once**,
and storing
(caching) the
result to avoid
recomputation.

The problem that created it: Naive recursion (Ch. 3) on problems like Fibonacci recomputes the same subproblems exponentially many times — `fib(5)` calls `fib(3)` twice, `fib(2)` three times, etc. —

$O(2^n)$ work for what's fundamentally an $O(n)$ amount of *distinct* work. Richard Bellman coined "Dynamic Programming" in the 1950s while working on multistage decision processes (resource allocation over time) — the core insight: **if a problem has optimal substructure (optimal answer built from optimal answers to subproblems) and overlapping subproblems (the same subproblem recurs), cache it.**

Real-life example:

Planning the cheapest multi-city flight itinerary — the cheapest way to reach city D via some route often reuses "the

cheapest way to reach city C," which many different routes to D also need. Instead of recomputing "cheapest way to reach C" every time it's needed, you compute it once, remember it, and reuse it — that's DP.

14.2 Core Theory

- **Optimal**

Substructure: the optimal solution to the problem can be constructed from optimal solutions to its subproblems. -

Overlapping

Subproblems: the same subproblems are solved repeatedly in a naive recursive solution (this is what distinguishes DP problems from plain divide-and-conquer like

Prev: [Heaps & Priority Queues](#) |

Next: [Chapter 13](#)
[— Graphs](#)

merge sort, where subproblems don't overlap). -

Two

implementation

styles: - **Top-**

Down

(Memoization):

write the natural recursive solution, add a cache (hashmap/array) keyed by subproblem parameters, check cache before recomputing. Easier to derive from brute force. -

Bottom-Up

(Tabulation):

identify the order subproblems must be solved in, fill a table iteratively from base cases up to the final answer. Usually faster in practice (no recursion overhead) and allows **space optimization** (often you only need the last 1-2 rows, reducing $O(n^2)$ space to $O(n)$ or $O(1)$). -

Prev: [Heaps & Priority Queues](#) |
Next: [Chapter 13](#)
[— Graphs](#)

State: what parameters uniquely define a subproblem (e.g., $dp[i]$ = best answer using first i items; $dp[i][j]$ = best answer for first i items with capacity j). -

Transition: the recurrence relating a state to smaller states.

14.3 Time & Space Complexity

Generally: **Time** = **(number of distinct states) × (time per transition)**. **Space** = **number of distinct states** (often reducible).

| DP Category |
Typical States |
Typical Time |
Typical Space
(optimized) | |—|
—|—|—| | 1D DP
(Fibonacci, House Robber, Climbing Stairs) | $O(n)$ | $O(n)$
| $O(1)$ | | Knapsack-style (2D) | $O(n \cdot W)$
| $O(n \cdot W)$ | $O(W)$ | |

Prev: [Heaps & Priority Queues](#) |
Next: [Chapter 13 — Graphs](#)

String matching
(LCS, Edit Distance) | $O(n \cdot m)$
| $O(n \cdot m)$ |
 $O(\min(n, m))$ | |
Interval DP (matrix chain, burst balloons) | $O(n^2)$
states | $O(n^3)$ |
 $O(n^2)$ | | Bitmask DP (TSP-style) |
 $O(n \cdot 2^n)$ states |
 $O(n^2 \cdot 2^n)$ | $O(n \cdot 2^n)$ |
| Tree DP | $O(n)$ states | $O(n)$ | $O(n)$ (recursion stack) |

14.4 The Big DP Patterns
(memorize these shapes)

1. 0/1 Knapsack
— choose items (each once) to maximize value under a weight constraint.
Template for:
subset sum, equal sum partition, target sum, minimum subset sum difference. 2.
Unbounded Knapsack — items can be reused. Template for: coin change

(min coins / count ways), rod cutting.

3. Longest

Common

Subsequence

(LCS) family —

two-sequence matching.

Template for: edit

distance, longest

palindromic

subsequence,

shortest common

supersequence,

distinct

subsequences. 4.

Longest

Increasing

Subsequence

(LIS) — $O(n^2)$ DP

or $O(n \log n)$ with

binary search +

patience sorting.

5. Kadane's / Max

Subarray style —

running local-vs-

global optimum,

$O(n)$. 6.

Matrix/Grid DP

— paths, min/max

path sum,

obstacles —

`dp[i][j]`

depends on

`dp[i-1][j]` and

`dp[i][j-1]`. 7.

Interval DP —

`dp[i][j]` = best

answer over subarray/substring $[i, j]$, built from smaller intervals inside — matrix chain multiplication, burst balloons, palindrome partitioning. 8.

Bitmask DP — state includes a bitmask representing a subset (which cities visited, which tasks done) — used when n is small ($\leq \sim 20$) and the state needs “which subset” info, e.g., Traveling Salesman Problem. 9. **Tree**

DP — DFS returning a value per subtree, combined at each node (House Robber III, Diameter). 10.

Digit DP — count numbers in a range satisfying a digit-based property, processing digit

Prev: [Heaps & Priority Queues](#) |
Next: [Chapter 13 — Graphs](#)

by digit with
tight/loose
bounds.

Code
skeleton — Top-
down
memoization
(Fibonacci)

Code
skeleton —
Bottom-up 0/1
Knapsack

14.5 Problems
(Basic → Medium
→ Hard)

Basic 1.
Climbing Stairs
(LeetCode 70) —
 $O(n)/O(1)$. 2.
House Robber
(LeetCode 198) —
 $O(n)/O(1)$. 3.
Fibonacci with
memoization vs
tabulation. 4. Min
Cost Climbing
Stairs (LeetCode
746). 5. Coin
Change (minimum
coins) (LeetCode
322).

Medium 6.
Coin Change II
(count ways)
(LeetCode 518) —

unbounded
knapsack. 7.

Longest

Increasing

Subsequence

(LeetCode 300) —

$O(n^2)$ then

optimize to $O(n$

$\log n$). 8. **Longest**

Common

Subsequence

(LeetCode 1143)

— $O(n \cdot m)$. 9. **Edit**

Distance

(LeetCode 72) —

$O(n \cdot m)$. 10. **0/1**

Knapsack /

Partition Equal

Subset Sum

(LeetCode 416).

11. **Target Sum**

(LeetCode 494).

12. **Unique Paths**

I & II (LeetCode

62, 63) — grid DP.

13. **Minimum**

Path Sum

(LeetCode 64). 14.

Word Break

(LeetCode 139).

15. **Decode Ways**

(LeetCode 91). 16.

House Robber II

(LeetCode 213) —

circular array

variant. 17.

Longest

Palindromic

Prev: [Heaps & Priority Queues](#) |
Next: [Chapter 13 — Graphs](#)

Subsequence

(LeetCode 516).

18. Maximal

Square (LeetCode 221) — grid DP.

Hard 19. **Edit**

Distance

variants: Distinct

Subsequences

(LeetCode 115).

20. Burst

Balloons

(LeetCode 312) — interval DP, $O(n^3)$.

21. Regular

Expression

Matching /

Wildcard

Matching

(LeetCode 10, 44).

22. Palindrome

Partitioning II

(LeetCode 132) — min cuts, interval DP. 23. **Best Time**

to Buy and Sell

Stock III & IV

(LeetCode 123, 188) — state machine DP with transaction limits.

24. Cherry Pickup

(LeetCode 741) — 2-agent grid DP.

25. Traveling

Salesman

Problem —

Prev: [Heaps & Priority Queues](#) |
Next: [Chapter 13 — Graphs](#)

Bitmask DP,
 $O(n^2 \cdot 2^n)$. 26.

Shortest

Common

Supersequence

(LeetCode 1092).

27. **Matrix Chain**

Multiplication

(classic GfG) —
interval DP. 28.

Count of

Numbers in

Range with digit

property — Digit
DP.

Prev: [Graphs](#) | **Next:** [Chapter 15 — Greedy Algorithms](#)

Chapter 15 — Greedy Algorithms

15.1 What & Why

A **Greedy Algorithm** builds a solution step by step, always choosing the option that looks best *right now*, without reconsidering past choices — and this happens to yield the global optimum for a specific class of problems.

The problem that created it: DP correctly explores/caches all relevant subproblem choices, but that's overkill when a problem has a special structure where the **locally optimal choice is always safe** (never needs to be undone). Making change with standard currency denominations, scheduling non-overlapping jobs, or building a minimum spanning tree can be solved greedily, avoiding DP's extra bookkeeping and getting simpler, often faster algorithms. Greedy was formalized by recognizing **matroid structure** and **exchange arguments** in optimization theory — proving that a greedy choice never eliminates the possibility of an optimal solution.

Real-life example: Making change for ₹87 using coins of ₹1, ₹2, ₹5, ₹10, ₹20 — you greedily take the largest coin that fits (₹20, ₹20, ₹20, ₹20, ₹5, ₹2), never needing to “undo” a coin choice, and this happens to be optimal for this denomination system (though NOT for all coin systems — this is exactly why greedy needs *proof*, not just intuition).

15.2 Core Theory

- **Greedy Choice Property:** a globally optimal solution can be reached by making a locally optimal (greedy) choice at each step.
- **Optimal Substructure** (shared with DP): the optimal solution contains optimal solutions to subproblems.
- **The critical difference from DP:** greedy makes ONE choice per step and never looks back or reconsiders; DP considers multiple choices and combines them, because in DP problems the “best now” choice is NOT always safe.
- **Proving greedy correctness** (interview-relevant): typically via an **exchange argument** — show that any optimal solution can be transformed into the greedy solution without making it worse, or via **induction** on the greedy choice being extendable to a full optimal solution.
- **Warning:** greedy does NOT always work — e.g., 0/1 Knapsack is NOT solvable greedily (must use DP), while Fractional Knapsack IS (take items by best value/weight ratio) — the difference (divisibility) determines whether the greedy choice is truly safe.

15.3 Time & Space Complexity

Greedy algorithms are typically fast because they make a single pass (often after sorting):

Pattern	Typical Time	Typical Space
Sort + single pass greedy	$O(n \log n)$	$O(1)$ to $O(n)$
Greedy + heap (priority-based decisions)	$O(n \log n)$	$O(n)$
Greedy + Union-Find (Kruskal’s MST)	$O(E \log E)$	$O(V)$

15.4 Key Patterns

1. **Activity Selection / Interval Scheduling** — sort by end time, greedily pick the earliest-ending compatible activity — maximizes count of non-overlapping intervals.
2. **Fractional Knapsack** — sort by value/weight ratio, take greedily until capacity fills.

3. **Huffman Coding** — build an optimal prefix-free encoding using a min-heap, always merging the two least-frequent nodes — classic greedy + heap combo (used in real compression algorithms).
4. **Interval Merging/Point Covering** — sort by start/end, sweep once.
5. **Two-Pointer Greedy** — e.g., gas station problem, candy distribution — one pass with a greedy local rule proven globally correct.
6. **Job Sequencing with Deadlines** — sort by profit, greedily assign to latest available slot before deadline.

15.5 Problems (Basic → Medium → Hard)

Basic

1. Activity Selection Problem (classic GfG) — sort by end time.
2. Fractional Knapsack (classic GfG) — sort by value/weight ratio.
3. Assign Cookies (LeetCode 455).
4. Lemonade Change (LeetCode 860).

Medium

5. **Jump Game I & II** (LeetCode 55, 45) — greedy reachability tracking, $O(n)$.
6. **Gas Station** (LeetCode 134) — one-pass greedy with proof via total-sum argument.
7. **Candy** (LeetCode 135) — two-pass greedy (left-to-right, right-to-left).
8. **Non-overlapping Intervals** (LeetCode 435) — sort by end, greedy count removal.
9. **Partition Labels** (LeetCode 763) — greedy interval merging using last-occurrence index.
10. **Task Scheduler** (LeetCode 621) — greedy + heap (also Ch. 11).
11. **Minimum Number of Arrows to Burst Balloons** (LeetCode 452) — sort by end, greedy.
12. **Queue Reconstruction by Height** (LeetCode 406) — sort + greedy insertion.

Hard

13. **Job Sequencing Problem with Deadlines** (classic GfG) — sort by profit + greedy slot assignment (often with Union-Find optimization).
14. **Huffman Encoding/Decoding** (classic GfG) — min-heap-based greedy construction.
15. **Minimum Platforms Required** (classic GfG, railway scheduling) — sort arrivals/departures, greedy sweep.
16. **Candy Distribution variants / IPO (Maximize Capital)** (LeetCode 502) — greedy + two heaps.
17. **Course Schedule III** (LeetCode 630) — greedy + max-heap.

Prev: [Dynamic Programming](#) | **Next:** [Chapter 16 — Bit Manipulation](#)

Prev: [Greedy Algorithms](#) | **Next:** [Chapter 17 — Advanced Data Structures](#)

Chapter 17 — Advanced Data Structures (Segment Tree, Fenwick Tree, DSU, LRU)

17.1 What & Why

Arrays give $O(1)$ point access but $O(n)$ range-sum/range-min queries; recomputing a sum over a range every query doesn't scale to millions of queries with interleaved updates. **Segment Trees** and **Fenwick Trees (Binary Indexed Trees)** were invented to support **both range queries AND point/range updates in $O(\log n)$** , which neither a plain array nor a

prefix-sum array
(fast query, but
 $O(n)$ update) can
do simultaneously.

Real-life example:

A stock-tracking dashboard needs “what’s the max price in the last 10 days?” (range query) recomputed continuously as new prices stream in (updates) — recalculating from scratch each time is too slow at scale; a Segment Tree answers both in $O(\log n)$.

17.2 Segment Tree

- **Structure:** a binary tree where each node represents a range $[l, r]$ of the array, storing an aggregate (sum/min/max/gcd) of that range; leaves are individual elements; each internal node = combine(left

child, right child). -

Build: $O(n)$. **Range**

Query: $O(\log n)$ —

decompose the query range into $O(\log n)$ tree nodes.

Point/Range

Update: $O(\log n)$ (or $O(\log n)$ with lazy propagation for range updates).

- **Lazy**

Propagation:

defers updates to child nodes until needed, enabling $O(\log n)$ **range updates** (e.g., "add 5 to every element in $[3,7]$ ") instead of $O(n)$.

Operation
Complexity — —
Build $O(n)$
Range Query
$O(\log n)$ Point
Update $O(\log n)$
Range Update (with lazy propagation)
$O(\log n)$ Space
$O(n)$ (typically $4n$ array size)

17.3 Fenwick Tree (Binary Indexed Tree, BIT)

- A more compact, simpler-to-code alternative to Segment Trees, but **limited to prefix-sum-like (invertible) operations** — sum, XOR — not min/max (which aren't invertible).
- Uses the binary representation of indices (i & $(-i)$) to find the next/parent index)
- to maintain implicit partial sums.
- Simpler, less memory, faster constant factor than Segment Tree — preferred when the operation is a sum/count/XOR.

Operation
Complexity — —
Build $O(n \log n)$ (or $O(n)$ with a smarter build)
Prefix Sum Query $O(\log n)$
Point Update $O(\log n)$
Space $O(n)$

Prev: [Dynamic Programming](#) |
Next: [Chapter 16](#)
— [Bit Manipulation](#)

Code skeleton
— Fenwick Tree

17.4 Disjoint Set Union (DSU / Union-Find) — recap & why it matters here

Already introduced in Ch. 13 (Graphs)
— grouped here because it's a general-purpose "advanced" structure beyond graphs: used for detecting cycles, dynamic connectivity queries, Kruskal's MST, and offline query processing tricks (e.g., "process queries in reverse to turn deletions into unions").

With **path compression + union by rank/size**, operations run in **$O(\alpha(n))$** amortized — α is the inverse Ackermann function, which is ≤ 4 for any practically

Prev: [Dynamic Programming](#) |
Next: [Chapter 16](#)
— [Bit Manipulation](#)

conceivable input
— effectively $O(1)$.

17.5 LRU/LFU
Cache Design
(recap link)

Covered in Ch. 6/8
— worth remembering here as the canonical “combine two data structures for $O(1)$ each op” advanced design pattern:
HashMap ($O(1)$ lookup) + Doubly Linked List ($O(1)$ reorder/evict).

17.6 Sparse Table (bonus, static range queries)

- For **immutable** arrays with range-min/max/gcd queries (no updates), a Sparse Table precomputes answers for all power-of-2-length ranges — $O(n \log n)$ build, **$O(1)$ query** (better than Segment Tree's $O(\log n)$, but only works when there are no updates).

17.7 When to use which (interview heuristic)

| Need | Structure | |
 —|—| | Range sum/min/max query + point update, need it general | Segment Tree | | Range sum/XOR query + point update, want simpler code | Fenwick Tree | | Range update + range query | Segment Tree with Lazy Propagation | | Static array, range min/max query, no updates | Sparse

Prev: [Dynamic Programming](#) |
Next: [Chapter 16](#) — [Bit Manipulation](#)

Table ($O(1)$ query) |
| Dynamic
connectivity / cycle
detection | Union-
Find (DSU) | | $O(1)$
get/put with
eviction policy |
HashMap + Doubly
Linked List
(LRU/LFU) |

17.8 Problems
(Basic → Medium →
Hard)

Basic 1. Build a
Fenwick Tree;
support prefix sum
query + point
update. 2. Build a
Segment Tree;
support range sum
query + point
update. 3.
Implement Union-
Find with path
compression +
union by rank.

Medium 4.
Range Sum Query
- Mutable
(LeetCode 307) —
Segment Tree or
Fenwick Tree. 5.
Count of Smaller
Numbers After
Self (LeetCode 315)

— Fenwick Tree
(also solvable via
merge sort, Ch. 5).

6. **Number of Islands II**

(LeetCode 305) —
Union-Find with
dynamic additions.

7. **Redundant Connection**

(LeetCode 684) —
Union-Find (also
Ch. 13).

8. **LRU Cache** (LeetCode
146) — HashMap +
Doubly Linked List.

9. **LFU Cache**
(LeetCode 460) —
HashMap +
frequency-
bucketed Doubly
Linked Lists.

Hard 10.

Range Sum Query 2D - Mutable

(LeetCode 308) —
2D Binary Indexed
Tree.

11. **The Skyline Problem**

(LeetCode 218) —
segment tree /
sweep line + heap.

12. **Count of Range Sum**

(LeetCode 327) —
merge sort or

Prev: [Dynamic Programming](#) |
Next: [Chapter 16 — Bit Manipulation](#)

Fenwick Tree over prefix sums. 13. **My**

Calendar III

(LeetCode 732) —

Segment Tree with lazy propagation for range

increments. 14.

Falling Squares

(LeetCode 699) —

coordinate

compression +

Segment Tree. 15.

Design Excel Sum

Formula /

advanced DSU

with union-by-

size for dynamic

connectivity

queries.

Prev: [Bit Manipulation](#) | **Next:** [Chapter 18 — Patterns Cheatsheet](#)

Chapter 18 — Master Patterns Cheatsheet

This chapter is a **quick-recall index**: when you see a problem, match its shape to a pattern here, then jump to the relevant chapter for depth.

18.1 “If you see X in the problem → think Y”

Signal in the problem	Likely Pattern	Chapter
“sorted array” + find target/pair	Binary Search / Two Pointers	4, 1
“subarray” / “substring” + contiguous + constraint	Sliding Window	1, 2
“all pairs summing to...”	Two Pointers (sorted) or HashMap	1, 8
“contiguous subarray sum equals K”	Prefix Sum + HashMap	1, 8
“all permutations / combinations / subsets”	Backtracking	3
“next greater/smaller element”	Monotonic Stack	7
“sliding window maximum/minimum”	Monotonic Deque	7
“kth largest/smallest” / “top K”	Heap	11
“median of a stream”	Two Heaps	11
“merge K sorted lists/arrays”	Min-Heap	11
“shortest path, unweighted graph”	BFS	13
“shortest path, weighted, non-negative”	Dijkstra (Heap)	13, 11
“shortest path, negative weights / detect neg cycle”	Bellman-Ford	13
“all-pairs shortest path”	Floyd-Warshall	13
“minimum spanning tree”	Kruskal’s (Union-Find) or Prim’s	13, 17
“detect cycle”	DFS (directed) / Union-Find (undirected)	13, 17
“order tasks with dependencies”	Topological Sort	13
“connected components / grouping”	Union-Find or BFS/DFS	13, 17

Signal in the problem	Likely Pattern	Chapter
"grid / matrix traversal, flood fill, islands"	BFS/DFS on grid	13
"distance to nearest X from multiple sources"	Multi-source BFS	13
"autocomplete / prefix matching"	Trie	12
"word search with dictionary"	Trie + Backtracking	12, 3
"maximum XOR pair"	Binary Trie or Bitmask	12, 16
"counting subsets / include-exclude each item once"	0/1 Knapsack DP	14
"unlimited reuse of items"	Unbounded Knapsack DP	14
"comparing two strings/sequences"	LCS-family DP	14
"longest increasing run"	LIS DP ($O(n \log n)$ with patience sorting)	14
"min/max path in grid"	Grid DP	14
" $n \leq \sim 20$, subset states matter"	Bitmask DP	14, 16
"optimal parenthesization / merging cost"	Interval DP	14
"schedule with deadlines/intervals to maximize count"	Greedy (sort by end time)	15
"compress data / build optimal encoding"	Huffman (Greedy + Heap)	15, 11
"range sum/min query + updates"	Segment Tree / Fenwick Tree	17
" $O(1)$ get/put with eviction"	HashMap + Doubly Linked List	6, 8, 17
"find the one unique/odd element"	XOR	16
"check power of two / count bits"	Bit tricks	16

Signal in the problem	Likely Pattern	Chapter
"binary search but not on a sorted array — on 'possible answers'"	Binary Search on Answer	4

18.2 Complexity Cheat Table (all structures at a glance)

Structure	Access	Search	Insert	Delete	Space
Array (static)	$O(1)$	$O(n)$	—	—	$O(n)$
Dynamic Array	$O(1)$	$O(n)$	$O(1)$ amortized (end)	$O(n)$	$O(n)$
Linked List (singly)	$O(n)$	$O(n)$	$O(1)$ (given node)	$O(1)$ (given node)	$O(n)$
Stack / Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
HashMap/HashSet	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$
Binary Search Tree (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Heap	$O(1)$ (top only)	$O(n)$	$O(\log n)$	$O(\log n)$ (top)	$O(n)$
Trie	—	$O(m)$	$O(m)$	$O(m)$	$O(n \cdot m)$
Segment Tree	—	$O(\log n)$ query	$O(\log n)$	$O(\log n)$	$O(n)$
Fenwick Tree	—	$O(\log n)$ query	$O(\log n)$	—	$O(n)$
Union-Find	—	$O(\alpha(n))$	$O(\alpha(n))$	—	$O(n)$

18.3 Recursion Recurrence → Big-O Cheat Sheet (Master Theorem, informal)

For $T(n) = a \cdot T(n/b) + O(n^d)$: - If $d > \log_b(a) \rightarrow O(n^d)$ (work dominated by the “combine” step — e.g., merge sort’s merge). - If $d = \log_b(a) \rightarrow O(n^d \cdot \log n)$ (e.g., merge sort: $a=2, b=2, d=1 \rightarrow \log_2(2)=1=d \rightarrow O(n \log n)$). - If $d < \log_b(a) \rightarrow O(n^{\log_b a})$ (work dominated by the recursive branching — e.g., naive Fibonacci-like exponential recursion).

18.4 Interview Problem-Solving Framework (UMPIRE-style)

1. **Understand**: restate the problem, clarify constraints (input size n tells you the target complexity — see Ch. 0.4 table), ask about edge cases (empty input, duplicates, negatives).
2. **Match**: use the table above — what pattern does this resemble?
3. **Plan**: state your approach and complexity *before* coding.
4. **Implement**: write clean code, use helper functions, meaningful names.
5. **Review**: trace through an example by hand, check edge cases (empty, single element, all same, extremes).
6. **Evaluate**: state final time/space complexity, mention trade-offs/alternatives if asked.

Prev: [Advanced Data Structures](#) |
Next: [Chapter 19](#)
— [Study Roadmap](#)

Chapter 19 —
Study Roadmap
for High-Package
Placements (₹1 Cr
/ FAANG-tier)

19.1
Suggested
Sequence
(already the order
of this tutorial)

| Phase | Weeks |
Chapters | Goal | |
—|—|—|—| |
Foundations | 1–2
| Ch 0–3
(Complexity,
Arrays, Strings,
Recursion) |
Comfortable
writing bug-free
code fast; explain
Big-O instantly | |
Core Linear DS |
3–4 | Ch 4–8
(Searching,
Sorting, Linked
List, Stack/Queue,
Hashing) |
Recognize &
implement all
core patterns
from memory | |
Non-Linear DS |
5–7 | Ch 9–13
(Trees, BST, Heap,
Trie, Graphs) |

Comfortable with
all traversal +
graph algorithms
|| Algorithmic
Thinking | 8–10 |
Ch 14–16 (DP,
Greedy, Bit
Manipulation) |
Solve
medium/hard DP
from scratch in
30–40 min ||
Advanced / Polish
| 11 | Ch 17–18
(Advanced DS,
Patterns) |
Segment/Fenwick
trees, full pattern
recall || Mock
Interviews | 12+ |
Full mixed
practice |
Simulate real
interview
pressure & time
limits |

19.2 Problem-
count targets
(roughly, adjust
to your pace)

- Basic tier: ~80–
100 problems
total across all
topics — builds
muscle memory
for syntax and
base cases. -
Medium tier:
~150–200
problems — this
is where

interviews are actually won; most FAANG/high-package interview questions sit here. - Hard tier: ~40–60 problems — enough to not be caught off guard, and to demonstrate depth in interviews that probe further.

19.3 How to practice each problem (don't just "read the solution")

1. Attempt for 20–30 minutes unaided.
2. If stuck, look at the **pattern name only** (Ch. 18), then re-attempt.
3. If still stuck, read the approach (not code), re-derive the code yourself.
4. Implement from scratch, dry run on paper/whiteboard with a small example.
5. State time/space complexity out loud, as if in an

interview. 6.

Revisit after 3 days, 1 week, 3 weeks (spaced repetition) — re-solve without looking.

19.4

Company-tier problem-set mapping (general guidance)

- **Product-based / high-package (FAANG, Google, Amazon, high-paying startups):**

heavy emphasis on Graphs, DP, Trees, Design (LRU/LFU, rate limiters), and clean code + complexity articulation under time pressure.

Expect 2–4 rounds of pure DSA plus a system design round at senior levels.

- **Service-based / broader hiring:** emphasis on Arrays, Strings, basic DP, SQL, and OOP design alongside DSA.

- For a “1 Cr package” target specifically:

expect **multiple DSA rounds at Hard-tier difficulty**, a strong System Design round, and behavioral rounds assessing ownership/scale of past projects — DSA alone is necessary but not sufficient; pair this tutorial with System Design and CS fundamentals (OS, DBMS, Networks) study.

19.5 Daily habit suggestion

- 2 new problems/day minimum (1 medium + 1 hard, or 2 medium), consistently, beats occasional marathon sessions.
- 1 full mock interview per week starting Phase 4 (timed, explain out loud, no IDE autocomplete).
- Maintain a mistake log: every bug/wrong approach → note the *root cause*

(off-by-one?
wrong pattern?
misread
constraint?) —
review this log
weekly.

19.6 What
“Done” looks like
for this tutorial

- You can state, from memory, the time/space complexity of every operation in Ch. 18.3’s table.
- Given any new problem, you can identify 1–2 candidate patterns within 2 minutes using Ch. 18.1.
- You can implement BFS/DFS, binary search, merge sort, quicksort, a heap, a Trie, and Union-Find from scratch, without references.
- You’ve solved at least the Medium tier of every chapter’s problem list once, and revisited each Hard-tier list at least once.